PARTNERSHIP UPDATE

Milestone Update

What we built. Why. How. What's next.

For the peaq tech, DePIN, and partnership team

AXI MOBILITY LIMITED

Service-as-Software for fleet operations

Building Africa's mobility infrastructure on peaq

Prepared for:	peaq tech / DePIN / partnership team
Pilot context:	ROAM Electric — 10 internal EV motorcycles, Nairobi
Document version:	1.0
Date:	May 7, 2026
Reference:	AXI/2026/PEAQ/001
Frozen tag:	`peaq-review-2026-05-07`
Contact:	team@aximobility.com

Apache-2.0 open source · Repos: `peaq-contracts` + `peaq-integration`

1 Executive summary

AXI Mobility runs a fleet platform. AI agents do the work of monitoring vehicles, drivers, energy, commands, and risk; we turn those signals into cases, actions, and reports. Customer is the supervisor. AXI is the operator.

1.1 What we built on peaq

Three production-shape Solidity contracts (Insurance Risk Oracle, High-Risk Command Vault, Credential Revocation Registry), all live on Agung testnet. Plus a TypeScript SDK and standalone CLI that wraps peaq's three pallets (`peaqDid`, `peaqRbac`, `peaqStorage`), peaq EVM via viem, KMS-backed Ed25519 signers, and a daily Merkle anchor service.

1.2 Status today

Where we are

- **51 unit and fuzz tests passing** on the local Foundry harness.
- **Agung deploy dry-run successful.** Production deploy runbook ready (`AGUNG_DEPLOY.md`) — 25 minutes from a blank `.env` to first attestation.
- **Pre-audit hardening checklist about 80% closed** (`AUDIT_CHECKLIST.md`). Remaining items: Echidna invariant runs, multisig setup, external audit firm engagement, `audit-v1` freeze tag.
- **Mainnet promote** is gated on Safe 4-of-7 multisig admin and external audit acceptance. Realistic window: 6–10 weeks from audit kickoff.

2 Why peaq

We chose peaq over generic Layer 1s and Layer 2s for one reason: **mobility-native chain primitives**.

2.1 The decision matrix

What we needed	What peaq gives	Generic-L1 alternative cost
Verifiable identity per vehicle and driver	`peaqDid` pallet (DID method spec, attribute extrinsics)	Build our own DID method and registry contract
On-chain RBAC across operators and regulators	`peaqRbac` pallet	Build OZ-AccessControl-shape registry + maintain it
Anchor and verify chain-of-custody for compliance	`peaqStorage` + EVM contracts side-by-side	Pick one, or run two chains
Realistic gas and finality for African DePIN	Substrate finality + EVM precompiles in one node	Bridge fees and two-chain ops
Mobility ecosystem alignment	DePIN focus, ROAM-class peers	Generic finance and DeFi peers

Strategic lock

We are not bridging across two chains. **peaq is the chain.** Polygon is our *fallback* for the daily Merkle anchor only — redundancy, not primary. This decision lives in `POSITIONING.md` as a strategic lock, not a tactical call.

3 What we built (the three use cases)

Each contract solves a real problem we ran into during the ROAM pilot design.

3.1 Insurance Risk Oracle

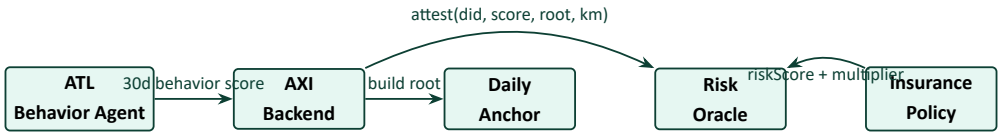
3.1.1 Problem

Insurance underwriters in Kenya quote premiums based on patchy fleet data. They charge a "risk surcharge" because they cannot trust the numbers. ROAM (and every fleet operator we talked to) overpays by 15–25%.

3.1.2 Solution

We attest a per-DID risk score on-chain. Append-only, signed by an authorised attestor. Score in $[0, 1000]$. Insurer reads the latest score before quoting. Verifiable. Auditable. Cheap.

3.1.3 End-to-end flow



3.1.4 Real ROAM scenario

ROAM rider Mary Akinyi

6 months riding · 4,200 km logged · 0 harsh-brake events · 89% safety score.

AXI attestor signs:

```
attest(did:peaq:mary, score=920, expiresAt=now+30d, payloadHash=0x...)
```

Insurer reads `getLatest(did:peaq:mary)` → score 920 → premium discount tier 1.

3.1.5 Why peaq vs a centralised database

Insurers will not trust AXI's database; we are an interested party. They will trust an on-chain attestation log signed by independent attestors. The chain gives them deniability: "we relied on a public register".

3.1.6 Implementation

Contract	`InsuranceRiskOracle.sol` (UUPS proxy, append-only history)
`ATTESTOR_ROLE`	AXI attestation service publishes scores
`CALIBRATOR_ROLE`	Risk team tunes the score-to-premium curve
`UPGRADER_ROLE`	Multisig 4-of-7 pushes upgrades
`DEFAULT_ADMIN_ROLE`	Multisig grants and revokes roles
Score range	$[0, 1000]$
Staleness gate	`riskScore()` reverts if > 7 days old

3.2 High-Risk Command Vault

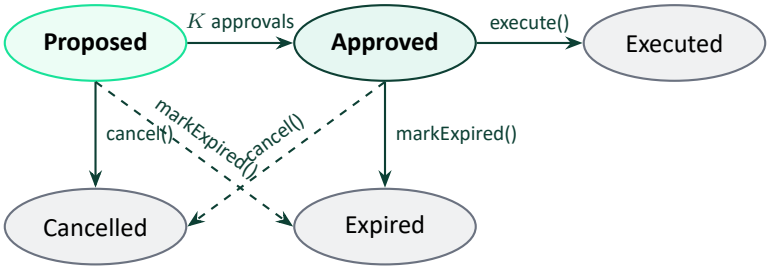
3.2.1 Problem

Fleet-wide actions are risky. "Lock 47 bikes" is one button, and it has to be reversible, auditable, and multi-party. Operator dashboards routinely have a single-sig escape hatch — that is how fleets get held ransom.

3.2.2 Solution

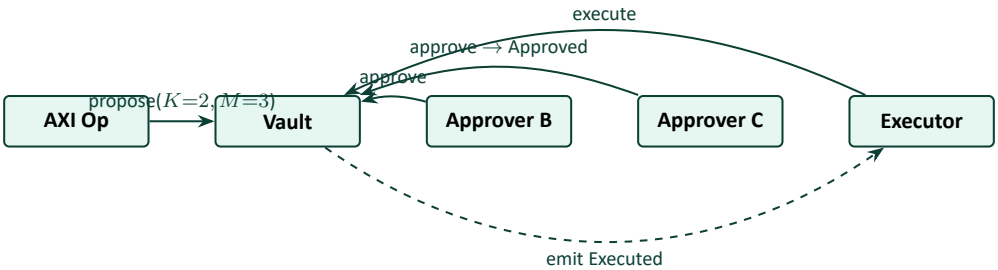
K-of-M approval gate. AXI staff propose a high-risk command (lock fleet, mass route override, broadcast more than 100 messages, fund withdrawal). M approvers must sign. Once K reach (out of M), the command becomes executable by anyone — including a regulator if AXI staff disappear.

3.2.3 State machine



Bright green = initial · Gray = terminal · Dashed = also reachable

3.2.4 End-to-end flow



3.2.5 Real ROAM scenario

Mary's bike pack-temp spike

KMG 117P shows pack temperature 52°C — risk of fire. AXI ops Brian proposes `LOCK_FLEET` on the 3 bikes in the same depot, $K = 2$ of $M = 3$.

- Joy approves · 1/2 reached
- ROAM ops manager approves · 2/2 reached → status flips to **Approved**
- Anyone calls `execute(commandId, payload)` → bikes locked

The on-chain audit shows: who proposed, who approved, when, and what.

3.2.6 Why peaq vs an off-chain Slack approval

A Slack approval can be repudiated. An on-chain K-of-M cannot. The audit trail is the product — insurance underwriters and regulators want to see operators are practising good governance before they price the premium or licence the fleet.

3.2.7 Implementation

Contract	`HighRiskCommandVault.sol` (**not upgradeable**)
Threshold range	$1 \leq K \leq M \leq 10$
Expiry window	1 minute to 30 days
Replay-safety	`commandId` hashes `(commandType, targetDid, payloadHash, K, M, nowTs, proposer)`
Payload binding	`propose()` stores `keccak256(payload)`; `execute()` must present same bytes
Permissionless execute	Once K reached, anyone can run it (no value extraction; documented MEV note)

Why not upgradeable

An upgradeable approval gate is a back door. The Vault is permanent — operators see exactly what they are approving and there is no path for AXI to silently change the rules.

3.3 Credential Revocation Registry

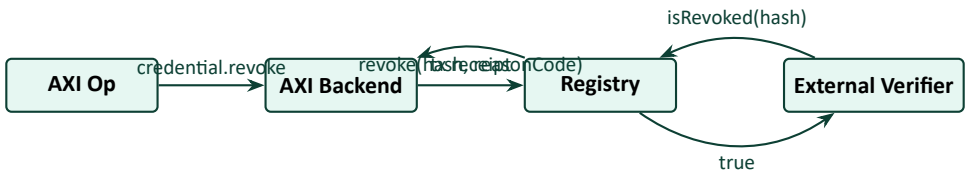
3.3.1 Problem

Driver licenses, NTSA inspection certificates, and insurance binders all expire or get revoked. Today fleets find out at the roadside check (about $300 fine plus a tow-truck bill). Insurers find out at claim time (about $5,000 loss). Verification is paper-based and manual.

3.3.2 Solution

Each credential is hashed and registered. When the issuer (NTSA, insurance company, AXI fleet manager) revokes, they call `revoke(hash, reasonCode)`. Anyone — insurer, regulator, the driver app itself — can check `isRevoked(hash)` for free. Per-issuer scope: only the issuer of a credential can revoke it.

3.3.3 End-to-end flow



3.3.4 Real ROAM scenario

ROAM rider Wanjiku's NTSA inspection

Inspection certificate expires 2026-05-15. NTSA officer revokes:

```
revoke(hash=0x... (Wanjiku's certificate), reasonCode=2 (expired))
```

At the next checkpoint, the inspector scans the QR code on Wanjiku's bike. The verify app calls `isRevoked(hash)` → **true**. Wanjiku is pulled aside, not jailed for "fraudulent docs". Saves AXI a court date.

3.3.5 Why peaq vs a central NTSA database

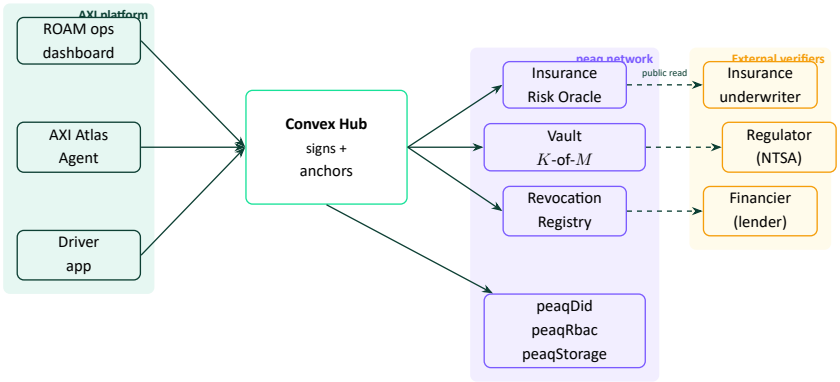
NTSA's portal goes down regularly. Our chain does not. Plus: insurance firms outside Kenya want to sell premium products to AXI fleets without trusting Kenya's central databases — chain-anchored revocation gives them an independent verification path.

3.3.6 Implementation

Contract	`CredentialRevocationRegistry.sol` (UUPS proxy)
`ISSUER_ROLE`	Per-credential issuer scope (only issuer of a credential can revoke it)
`revoke()`	Idempotent at boundary — re-revoke is rejected to preserve original `reasonCode`
`revokeBatch()`	Up to 100 revocations per transaction (handy for issuer-key rotation)
`unrevoke()`	Admin-only emergency function (in case issuer key is compromised)
`pause()`	Emergency stop

4 How it all fits together

4.1 The architecture



4.2 Three flows, every day

- **Daily Merkle anchor (00:00 UTC)** — Convex cron walks the audit log, builds a Merkle tree, anchors the root on peaq mainnet (Polygon as fallback). `peaqscan` and Subscan both show the same root; anyone can pick a random event from yesterday and walk the proof.
- **Per-event critical hash (live)** — ATL agent flags a high-risk event; KMS-backed Ed25519 signer submits a tx to peaq; case UI shows a verified pill within seconds.
- **Public verification (on demand)** — `verify.aximobility.com/api/verify/[hash]` returns a Merkle proof; verifier walks the proof against peaq directly. No trust in AXI required.

5 The peaq footprint we use

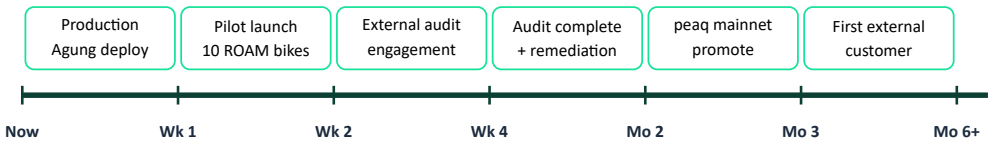
peaq feature	How AXI uses it	Frequency
EVM smart contracts	3 production contracts (Vault, Oracle, Registry)	1–10×/day
`peaqDid` pallet	DID issuance per vehicle, driver, asset	~50/day pilot
`peaqRbac` pallet	Cross-tenant role assignments (issuer, attestor, approver)	per role change
`peaqStorage` pallet	Selective evidence storage (compliance binders)	monthly
EVM precompiles	viem-compatible `defineChain` + standard tooling	every chain call
Mainnet RPC	Daily anchor + per-event critical hash	continuous
Agung testnet	Pre-mainnet integration testing	continuous in CI

6 Milestones reached

Date	Milestone	Status
2026-04	Lock peaq as primary chain (Polygon as fallback) — strategic decision in `POSITIONING.md`	✓
2026-04	Verified 34 peaq facts + flagged 7 open questions in `VERIFIED_SPECS.md`	✓
2026-04	Built TypeScript integration: EVM client (viem), Substrate client, DID issuer/resolver, RBAC + Storage pallet wrappers, KMS-Ed25519 signers, idempotency, retry, structured logger	✓
2026-04	Built standalone `peaq-cli` ops tool with 5 command groups (anchor, did, storage, health, status)	✓
2026-04	Wrote 3 production-shape Solidity contracts with UUPS, OZ AccessControl, custom errors, NatSpec	✓
2026-04	51 unit + fuzz tests passing on local Foundry harness	✓
2026-05	Slither in CI severity-gated to high; `forge fmt` + snapshot in CI	✓
2026-05	Agung deploy dry-run successful (chain ID 9990)	✓
2026-05	`AUDIT_CHECKLIST.md` (about 80% items closed)	✓
2026-05	`AGUNG_DEPLOY.md` 25-minute zero-to-first-attestation runbook	✓
2026-05	Per-contract STRIDE threat models published in `threat-models/`	✓
2026-05	`SECURITY.md` responsible disclosure policy	✓
2026-05	`PEAQ_REVIEW.md` hand-off packet for peaq tech team	✓
2026-05	This milestone document	✓

7 Roadmap

7.1 Visual timeline



7.2 Roadmap detail

Milestone	When	What it unlocks
Production Agung deploy	Week 1	Live testing against real Agung; pilot can chain-anchor daily
Internal pilot launch	Week 2	First chain anchor, first risk attestation, first revocation
External audit engagement	Week 3–4	Lock contracts, tag `audit-v1`
Audit + remediation	Week 4–8	Gate to mainnet
peaq mainnet promote	Month 2	Daily anchor live, Safe 4-of-7 multisig admin
First external customer	Month 3	Same platform, different ontology
Bug bounty live	Month 3	Phase 5 gate per `EXECUTION_ORDER.md`
First sovereign customer deploy	Month 6+	Atlas Sovereign K2 LoRA on AWS GPU

8 What we'd love from the peaq team

8.1 Direct answers to seven unverified questions

These come from `VERIFIED_SPECS.md`. We want to publish answers in that file as "verified, source: peaq team confirmation".

Capacity + economics

1. **Exact extrinsic fees.** Cost of `peaqDid.addAttribute` and `peaqStorage.addItem` in PEAQ + USD?
2. **DePIN bulk-write pricing.** Is there a tier for high-frequency writes? At ROAM Phase 4 (∼10,000 vehicles), we will write 10–100 events per second.

Compliance + identity

3. **W3C JSON-LD VC compliance.** Does peaq DID resolve correctly inside W3C VC libraries? The insurance arbitrage thesis depends on insurers consuming our DIDs in their existing tooling.

Network safety

4. **Decentralisation metrics.** Validators and collators count, geographic distribution, nakamoto coefficient.
5. **Major incidents and forks since mainnet launch.** For our risk model.
6. **KREST → PEAQ migration plan.** What is the canary network policy? Is there guidance for partners?

Tooling

7. **Official viem chain definition.** Does peaq plan to ship one? We are synthesising via `defineChain` today.

8.2 Code review

- Validate our `did:peaq:...` issuer and resolver against the spec repo.
- Validate our pallet wrapper code (storage and RBAC).
- Spot-check the EVM contracts; flag anything peaq has seen go wrong in other DePIN deployments.

8.3 Mainnet promotion guidance

- Preferred audit firm list? (We are considering Trail of Bits, OpenZeppelin, Halborn, Sigma Prime.)
- Monitoring requirements before a partner goes mainnet?
- Multisig setup recommendations? (Safe on peaq EVM is the path; signers + threshold.)
- Public launch playbook — does peaq announce and amplify partner mainnet launches?

8.4 Partnership amplification

- Co-marketing for the ROAM mainnet promote — joint announcement with ROAM and a peaq team blog post.
- DePIN ecosystem listing — list AXI Mobility on peaq's DePIN partners page.
- Reference architecture write-up — happy to contribute a "how AXI uses peaq" reference for other DePIN partners.

Time-sensitive

The ROAM internal pilot launches in 2 weeks. We want to land on Agung mainnet for daily anchor before then. A 1-hour kickoff call this week unblocks our timeline. Reply to team@aximobility.com.

9 Next actions

9.1 For the peaq team

1. Read this milestone document (15-minute scan).
2. Read `PEAQ_REVIEW.md` in the repo if you want technical depth (90-minute deep dive).
3. Reply to team@aximobility.com with availability for a 1-hour kickoff call this week.
4. Send written answers to the seven questions in section 7.1, or come ready to discuss them on the call.

9.2 For AXI (in parallel)

- Production Agung deploy with funded EOA + multisig admin.
- Close the remaining 20% of `AUDIT_CHECKLIST.md` (Echidna, branch coverage, mutation testing).
- Engage external audit firm (preference among Trail of Bits, OpenZeppelin, Halborn, Sigma Prime — pending peaq team's recommendation).
- Tag `audit-v1` freeze commit before audit kickoff.

9.3 Repository links

peaq-contracts	github.com/aximobility/peaq-contracts
peaq-integration	github.com/aximobility/peaq-integration
Frozen tag	`peaq-review-2026-05-07`
Licence	Apache-2.0 (both repos)

9.4 Documents in this packet

File	Purpose
This document	Partnership-facing milestone update (you are here)
`MILESTONES.md`	Same content, GitHub-rendered Markdown
`PEAQ_REVIEW.md`	13-section technical review packet
`AUDIT_CHECKLIST.md`	Pre-audit hardening status (~80% closed)
`AGUNG_DEPLOY.md`	25-minute step-by-step deploy walkthrough
`SECURITY.md`	Responsible disclosure policy
`threat-models/*.STRIDE.md`	Per-contract STRIDE + DREAD analysis
`VERIFIED_SPECS.md`	34 verified peaq facts + 7 open questions

Document info

Document version:	1.0
Date:	May 7, 2026
Reference:	AXI/2026/PEAQ/001
Frozen tag:	`peaq-review-2026-05-07`
Repos:	`peaq-contracts` + `peaq-integration`
Contact:	team@aximobility.com
Website:	www.aximobility.com

AXI MOBILITY LIMITED

Nation Centre, Kimathi Street · P.O. Box 49010, G.P.O. Nairobi

+254 105 675 224 · www.aximobility.com